CS 214: Data Structures

Queues

Slide contents follow

Kruse and Leung "Data Structures & Program Design in C"

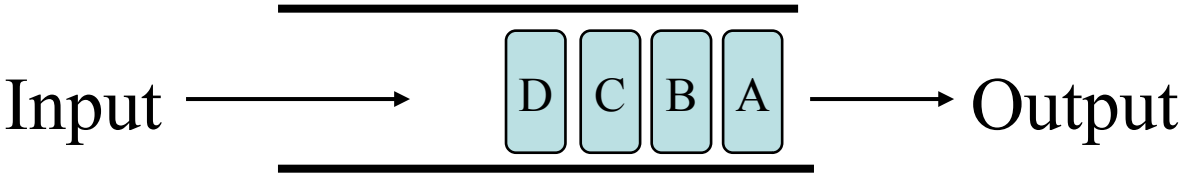
Prepared by:

Waleed A. Yousef, Ph.D.

© Waleed A. Yousef 2008

Motivation: Why `Queue`? *First In First Out (FIFO)*

- In a queue,
  - **new** values are always **added** at the rear of the list
  - values are **removed** from the opposite end of the list, the front

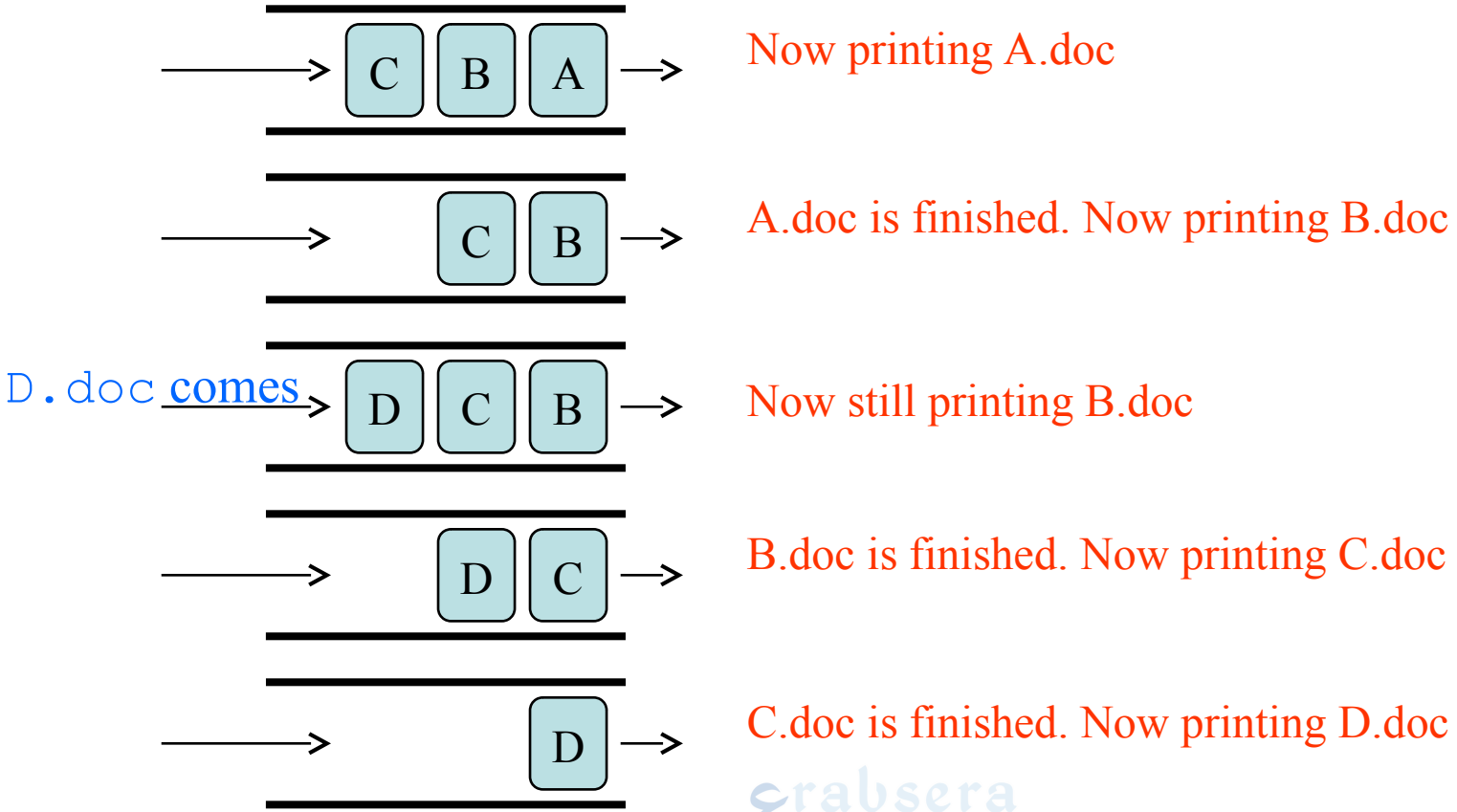


- **Examples of queues**
  - checkout at supermarket
  - Toll Station
    - Car comes, pays, leaves
  - Check-out in Big super market
    - Customer comes, checks out and leaves
  - More examples: Printer, Office Hours, …

© Waleed A. Yousef 2008

E.g., Printing Queue

- `A.doc, B.doc, C.doc` arrive to printer.

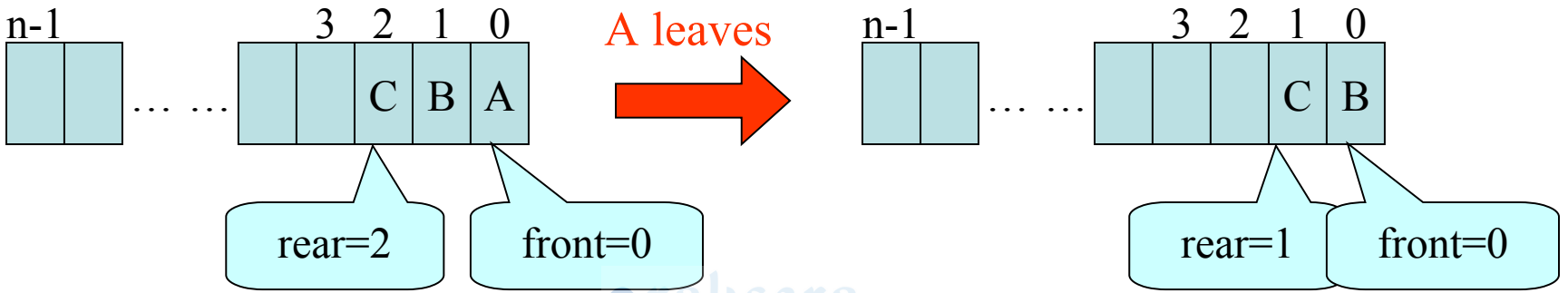


© Waleed A. Yousef 2008

Array Implementation1: Physical Model (Front is fixed as in physical lines)

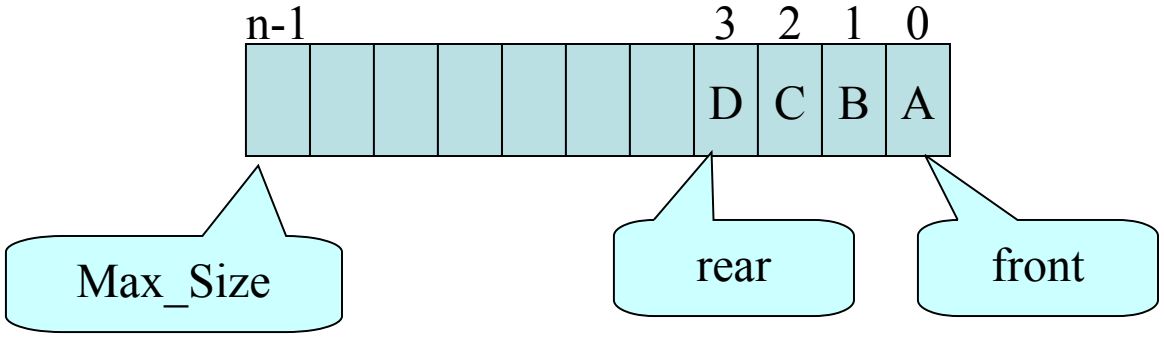
- Shifting all items to front in the array when dequeue operation. (Too Costly…)

- Why this was not a problem in the array implementation of Stacks?

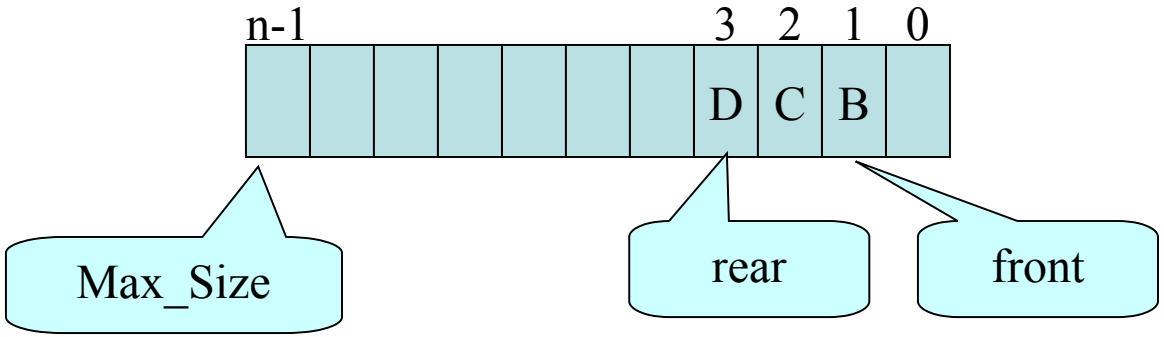


© Waleed A. Yousef 2008

Array Implementation2: Linear Model (Two indices, front and rear)

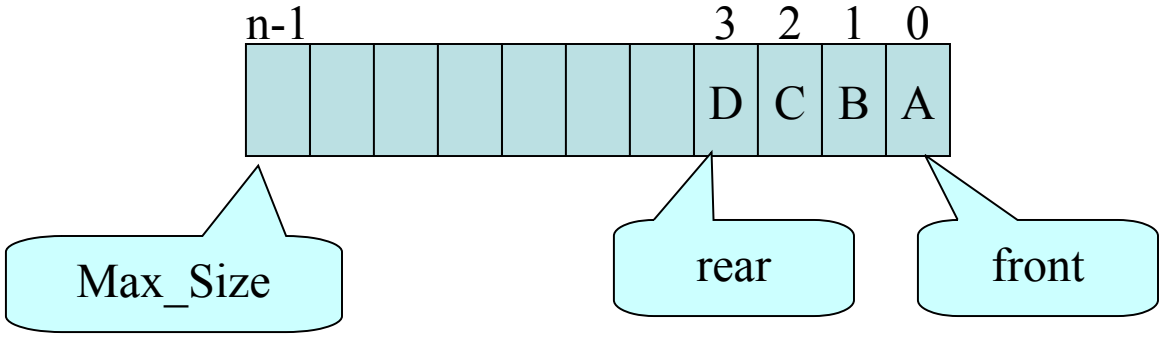


After A leaves,

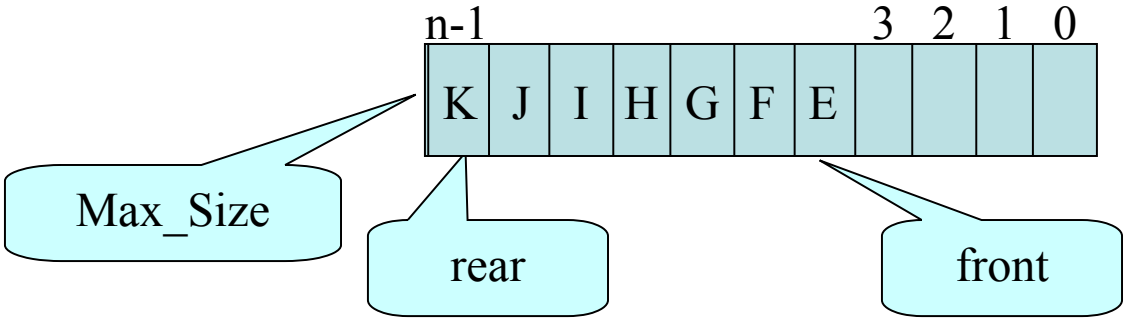


© Waleed A. Yousef 2008

The problem is that there will be many empty places in the front of array and rear is always incremented.

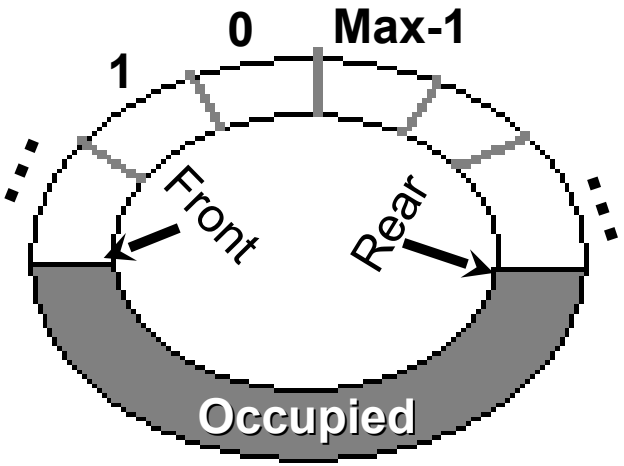


After A,B,C,D leave



© Waleed A. Yousef 2008

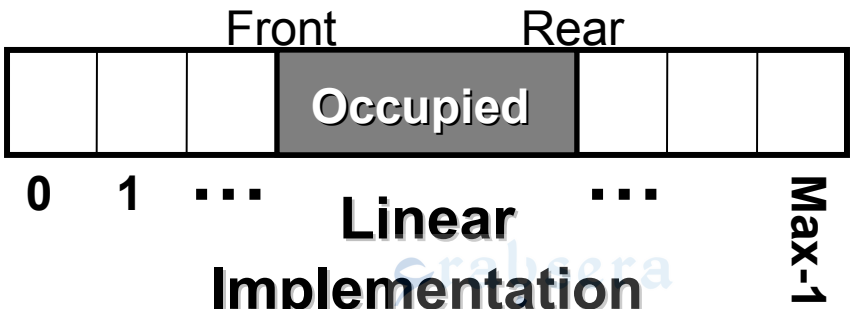
Array Implementation 3: Circular Implementation



Circular Queue



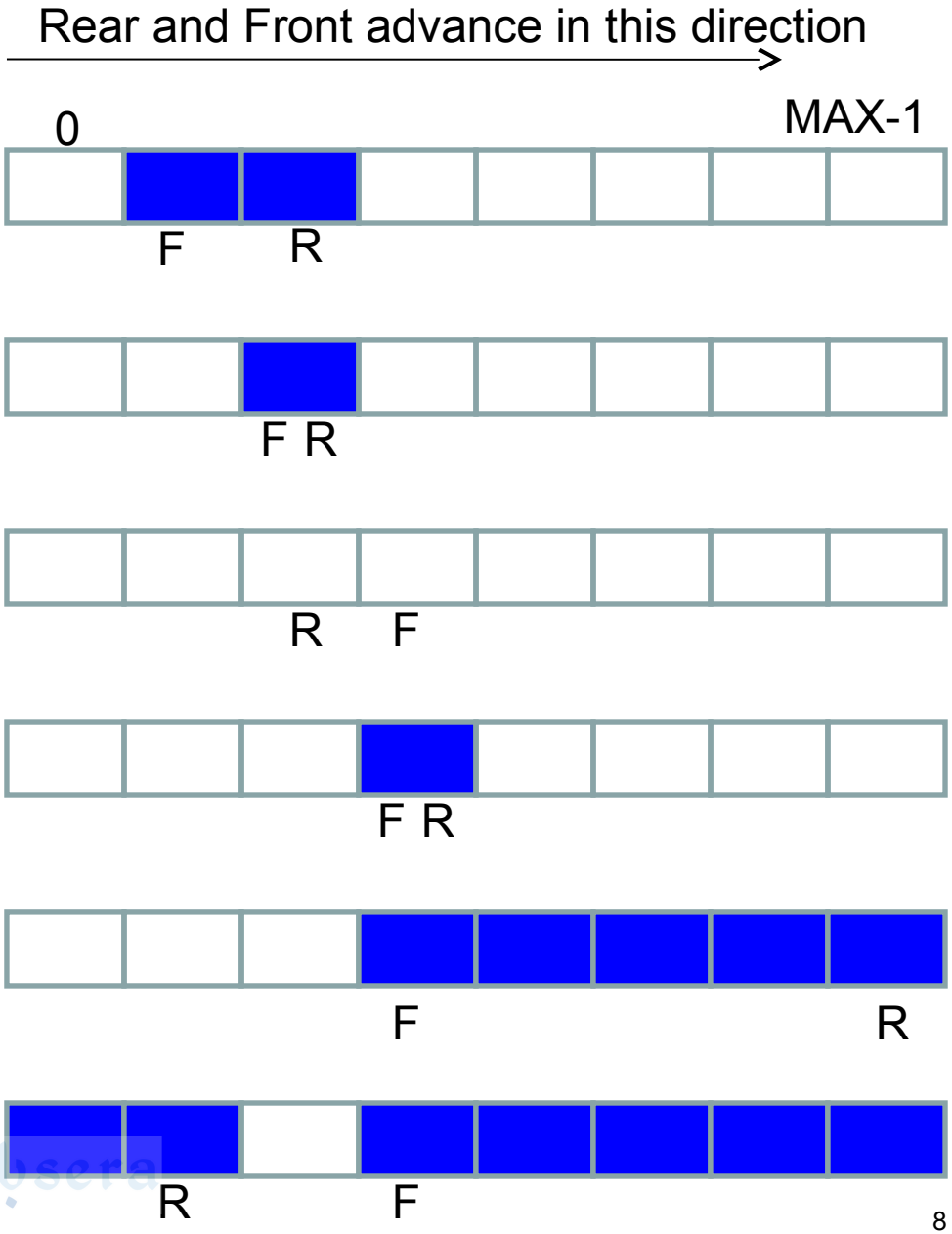
Unwinding



Linear Implementation

© Waleed A. Yousef 2008

Checking the Boundary conditions



© Waleed A. Yousef 2008

Better solution: Use indicator variable to distinguish between Empty and Full conditions.

Rear and Front advance in this direction

Empty queue:
Next to Rear = Front.
Size=0





Full queue:
Next to Rear = Front.
Size=MAX



© Waleed A. Yousef 2008

Again: Definitions, where every thing should start from!

Definition: ***type*** is a set of values and a set of operations on those values.

Example: We can define a datatype `boolean` that takes the set of values `{0, 1}` together with the operations `AND`, `OR`, and `NOT`.

Example: `int`, in `C`, is the set consisting all of the integers between `INT_MIN` $(-(2^{15} - 1))$ and `INT_MAX` $(2^{15} - 1)$, which are defined in the header file `limits.h`

Definition: A ***Sequence of length 0*** is empty. A ***sequence of length*** $n \geq 1$ of elements from a set `T` is an ordered pair (S_{n-1}, t) where S_{n-1} is a sequence of length $n-1$ of elements from `T`, and `t` is an element of `T`.

© Waleed A. Yousef 2008

Definition: A ***queue*** of elements of type T is a finite sequence of elements of T together with the following operations:

1. Create the queue, leaving it empty.
2. Determine whether the queue is empty or not
3. Determine whether the queue is full or not
4. Find the size of the queue
5. Append a new entry onto the top of the queue, provided the queue is not full.
6. Retrieve the front entry in the queue, provided the queue is not empty.
7. Serve (and remove) the front entry from the queue, provided the queue is not empty.
8. Traverse the queue, visiting each entry
9. Clear the queue to make it empty

© Waleed A. Yousef 2008

User Level (interface)

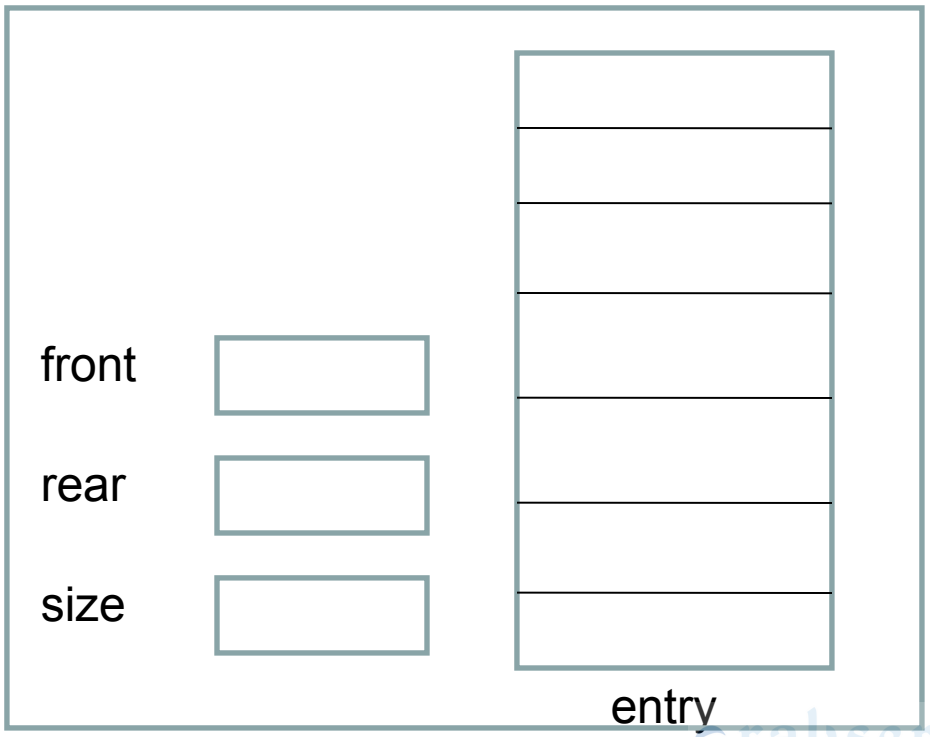
```
typedef struct queue{
    int front;
    int rear;
    int size;
    QueueEntry entry[MAXQUEUE];
}Queue;
```

```
void main(){

Queue q;



}
```



© Waleed A. Yousef 2008

User Level (interface)

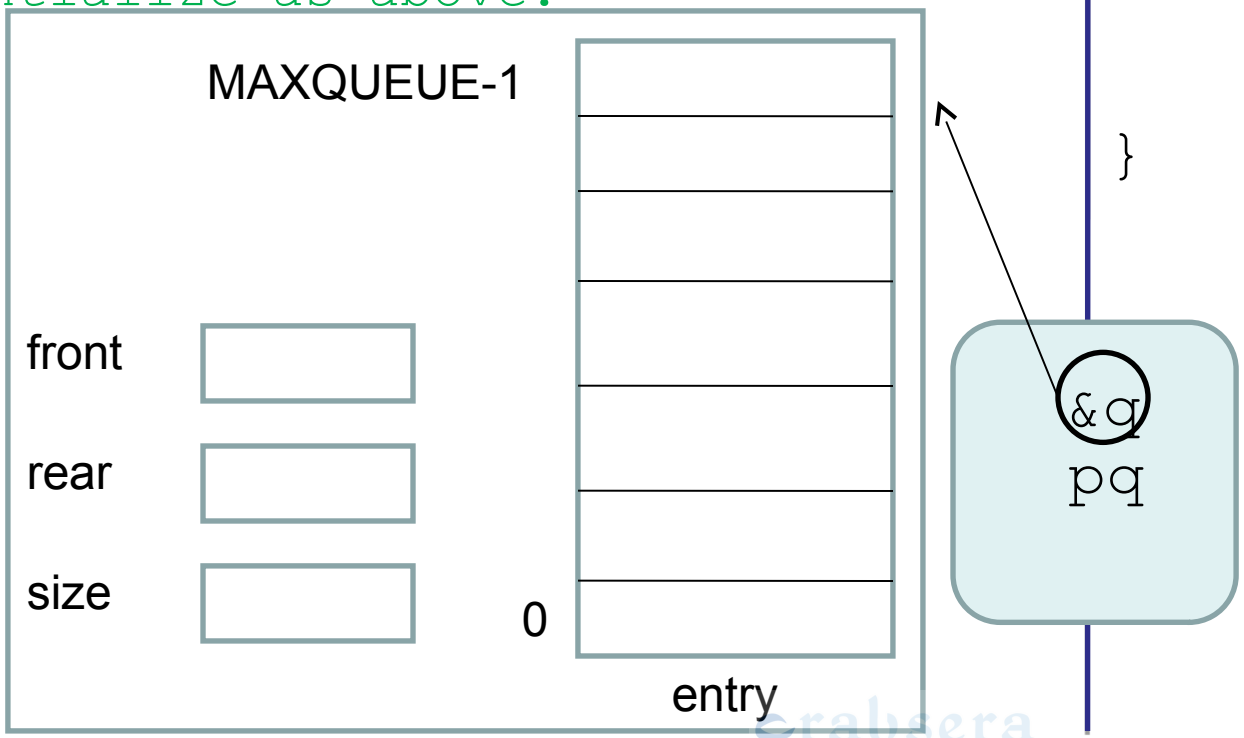
```
void CreateQueue(Queue *pq){
      pq->front= 0;
      pq->rear = -1;
      pq->size = 0;
}
//Initializing front =5 and rear
=4 will work if MAXQUEUE >=6. But,
since MAXQUEUE can be 1 we
intialize as above.
```

```
void main(){

Queue q;

CreateQueue(&q);

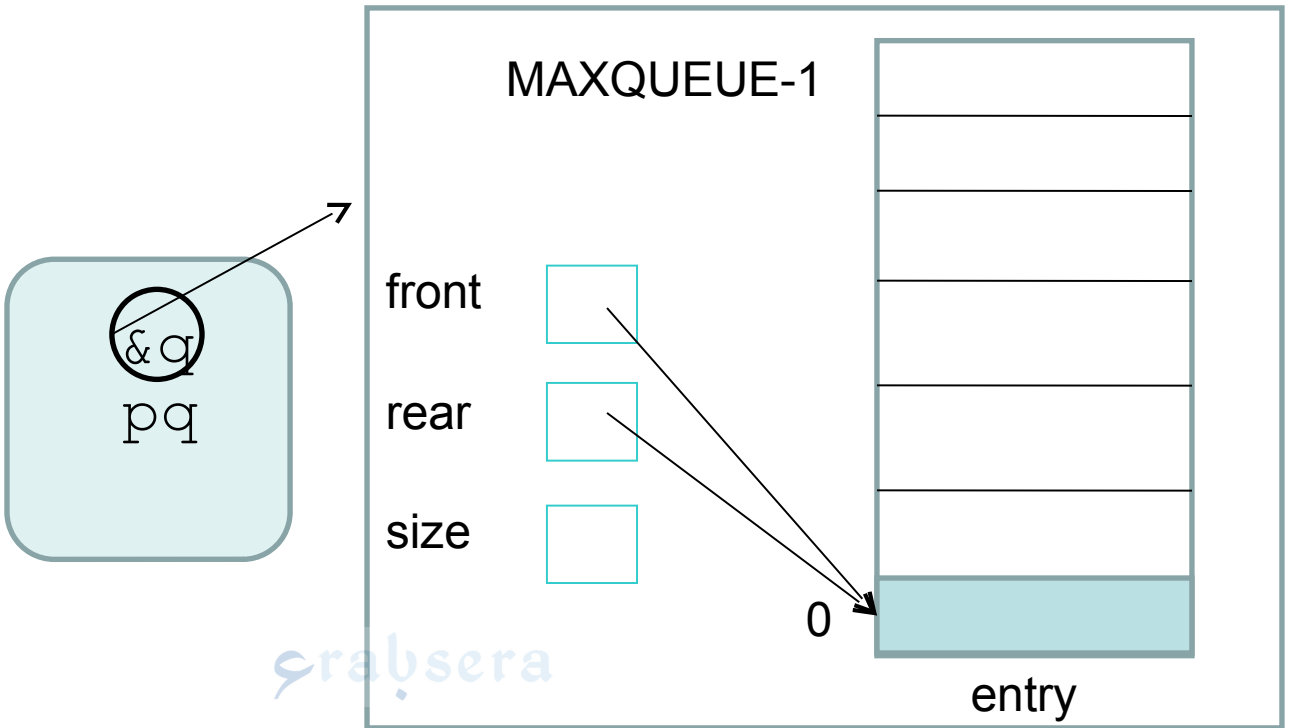

}
```



© Waleed A. Yousef 2008

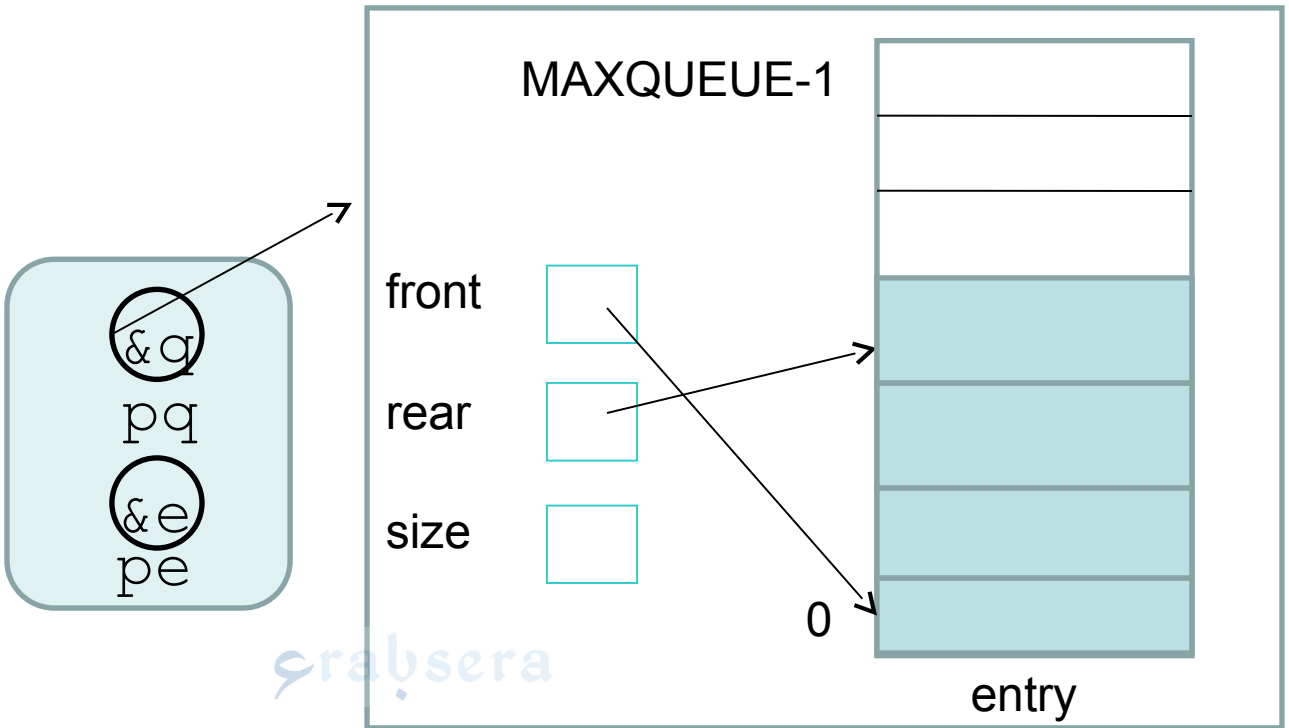
```
void Append(QueueEntry e, Queue* pq){
  pq->rear = (pq->rear + 1) % MAXQUEUE;
  pq->entry[pq->rear] = e;
  pq->size++;
}
```

```
if (pq->rear == MAXQUEUE-1)
      pq->rear=0;
else
      pq->rear++;
```



© Waleed A. Yousef 2008

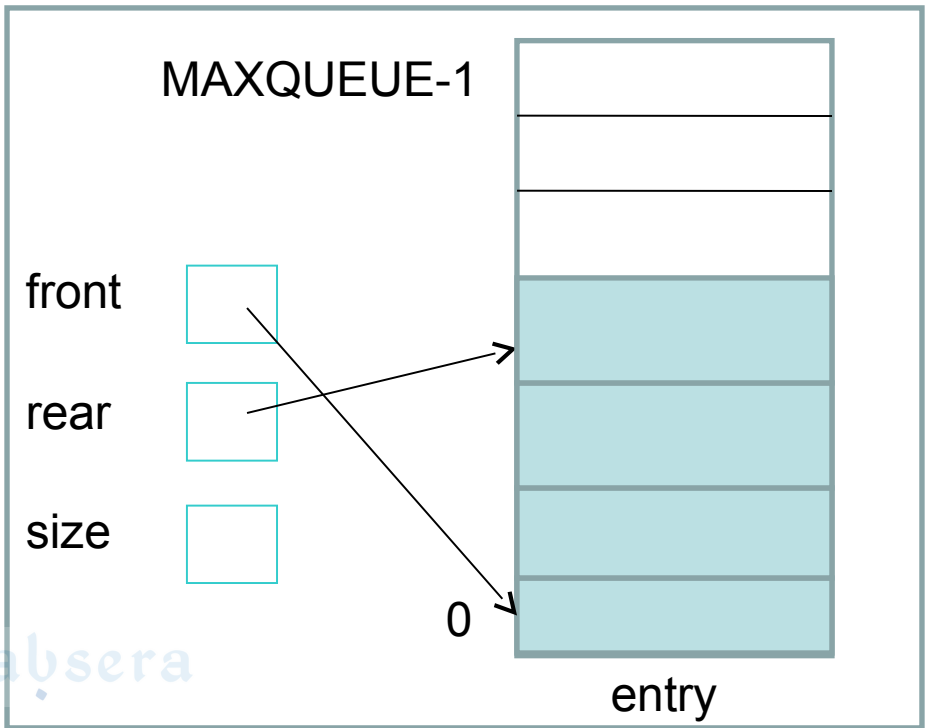
```
void Serve(QueueEntry *pe, Queue* pq){
   *pe = pq->entry[pq->front];
   pq->front = (pq->front + 1) % MAXQUEUE;
   pq->size--;
}
```



© Waleed A. Yousef 2008

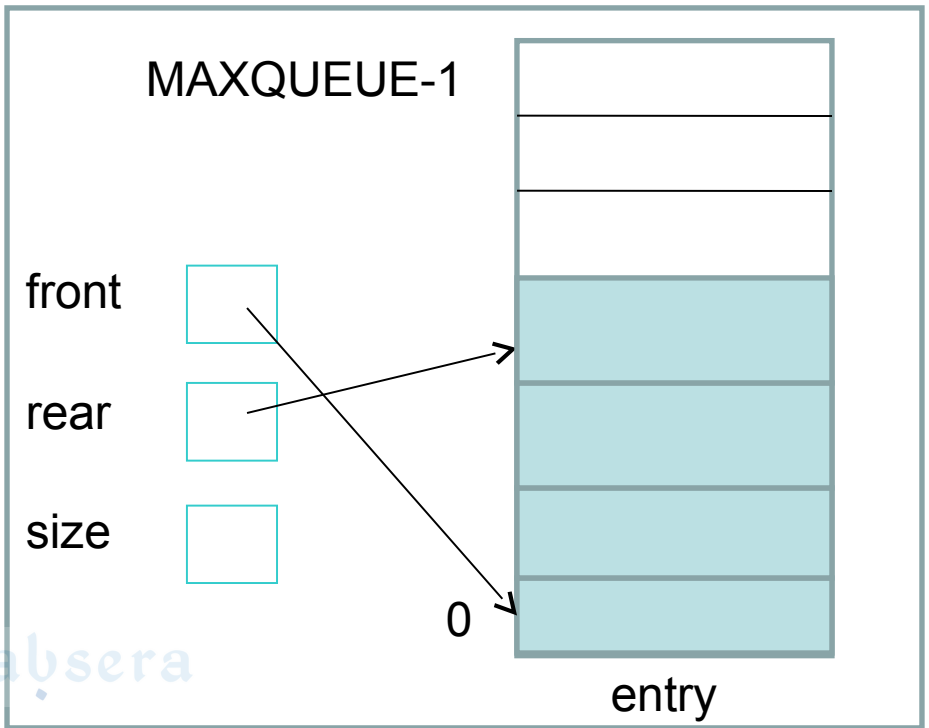
```
int QueueEmpty(Queue* pq){
     return !pq->size;
}

int QueueFull(Queue* pq){
     return (pq->size == MAXQUEUE);
}
```



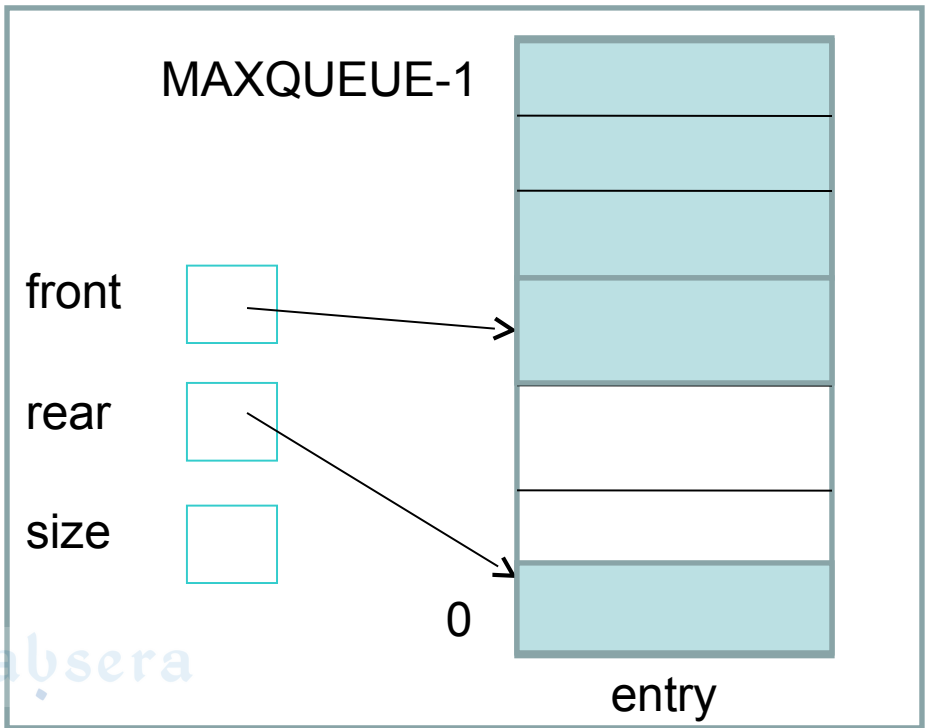
© Waleed A. Yousef 2008

```
int QueueSize(Queue* pq){
      return pq->size;
}
void ClearQueue(Queue* pq){
      pq->front = 0;
      pq->rear  = -1;
      pq->size  = 0;
}//same as CreateQueue. No nodes to free.
```



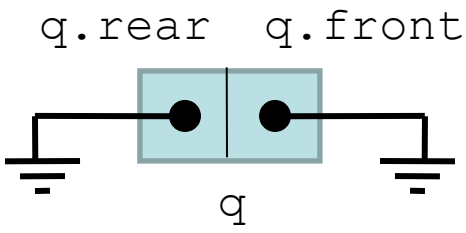
© Waleed A. Yousef 2008

```
void TraverseQueue(Queue* pq, void (*pf)(QueueEntry)){
      int pos, s;
      for(pos=pq->front, s=0; s<pq->size; s++){
            (*pf)(pq->entry[pos]);
            pos=(pos+1)%MAXQUEUE;
      }
}
```

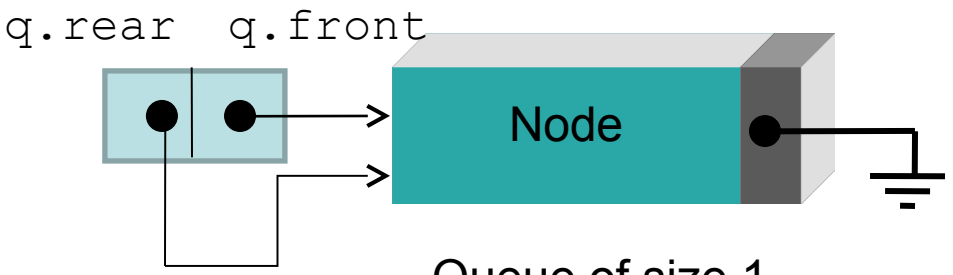


© Waleed A. Yousef 2008

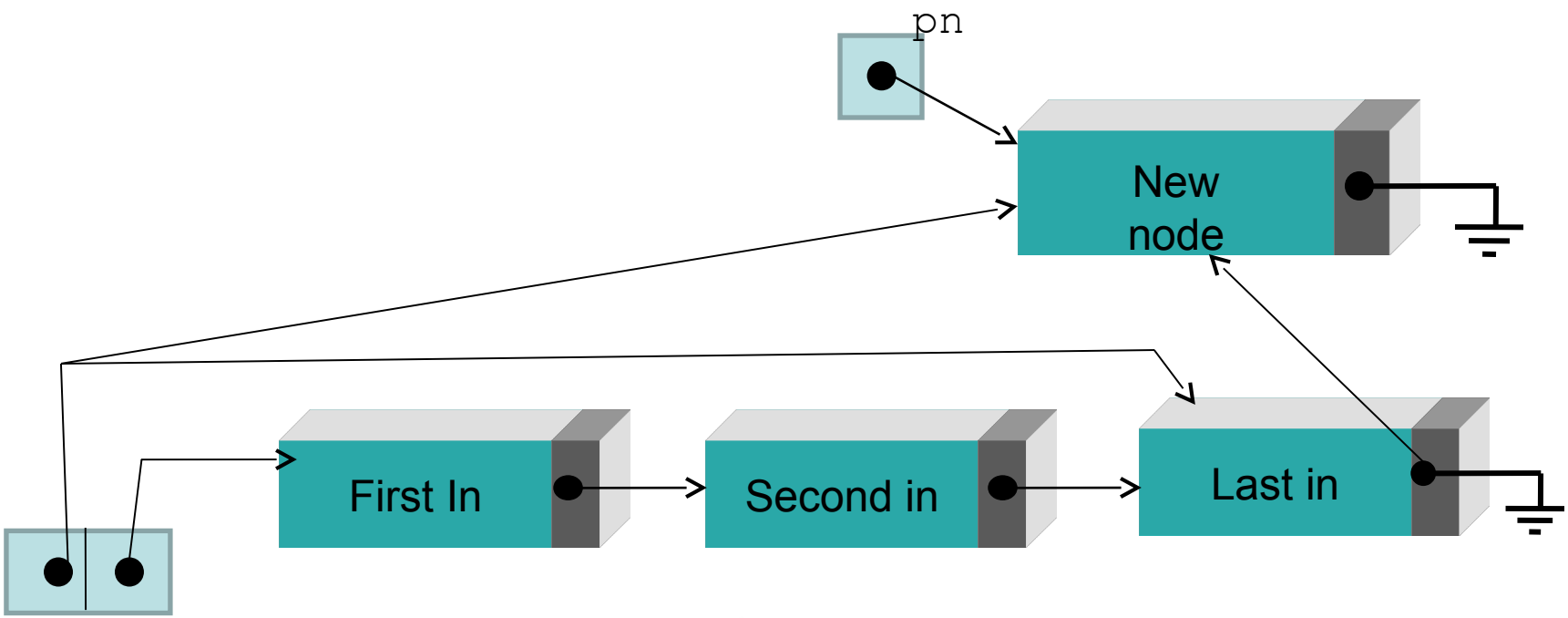
Linked Queues(to overcome fixed size limitations): Just get the idea now, do not worry about details.



Empty Queue



Queue of size 1

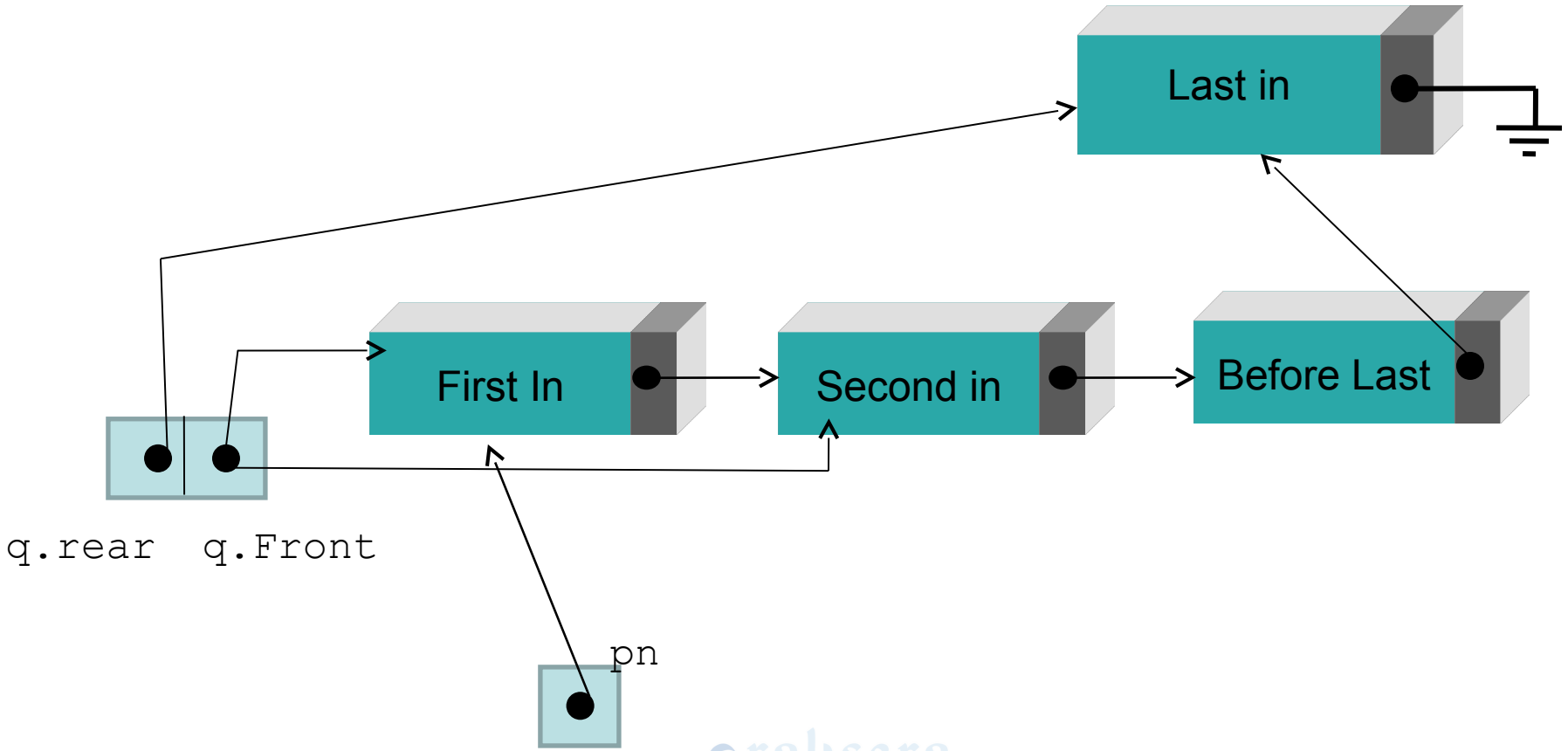


How to insert a node

© Waleed A. Yousef 2008

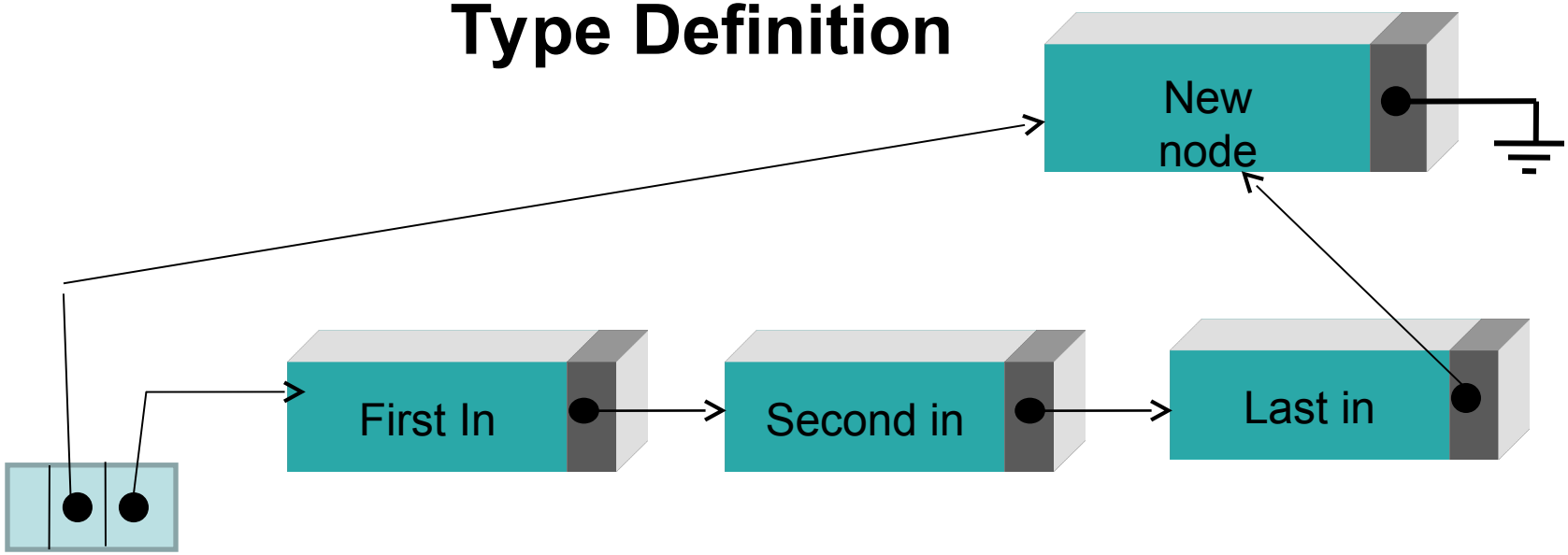
Linked Queues(to overcome fixed size limitations): Just get the idea now, do not worry about details.

How to serve a node



© Waleed A. Yousef 2008

Type Definition



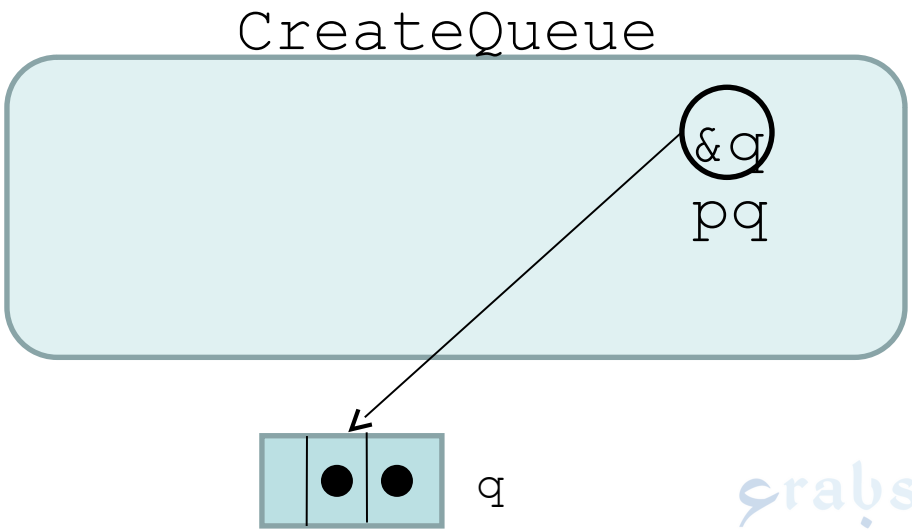

```
typedef struct queuenode{
      QueueEntry entry;
      struct queuenode *next;
}QueueNode;

typedef struct queue{
      QueueNode *front;
      QueueNode *rear;
      int  size;       //old trick
}Queue;
```

© Waleed A. Yousef 2008

Implementation level (what really happens)

```
void CreateQueue(Queue *pq){
    pq->front=NULL;
    pq->rear=NULL;
    pq->size=0;
}
```



User Level (interface)

```
void main(){

Queue q;

CreateQueue(&q);


}
```

© Waleed A. Yousef 2008

```
void Append(QueueEntry e, Queue* pq){
  QueueNode*pn=(QueueNode*)malloc(sizeof(QueueNode));
  pn->next=NULL;
  pn->entry=e;

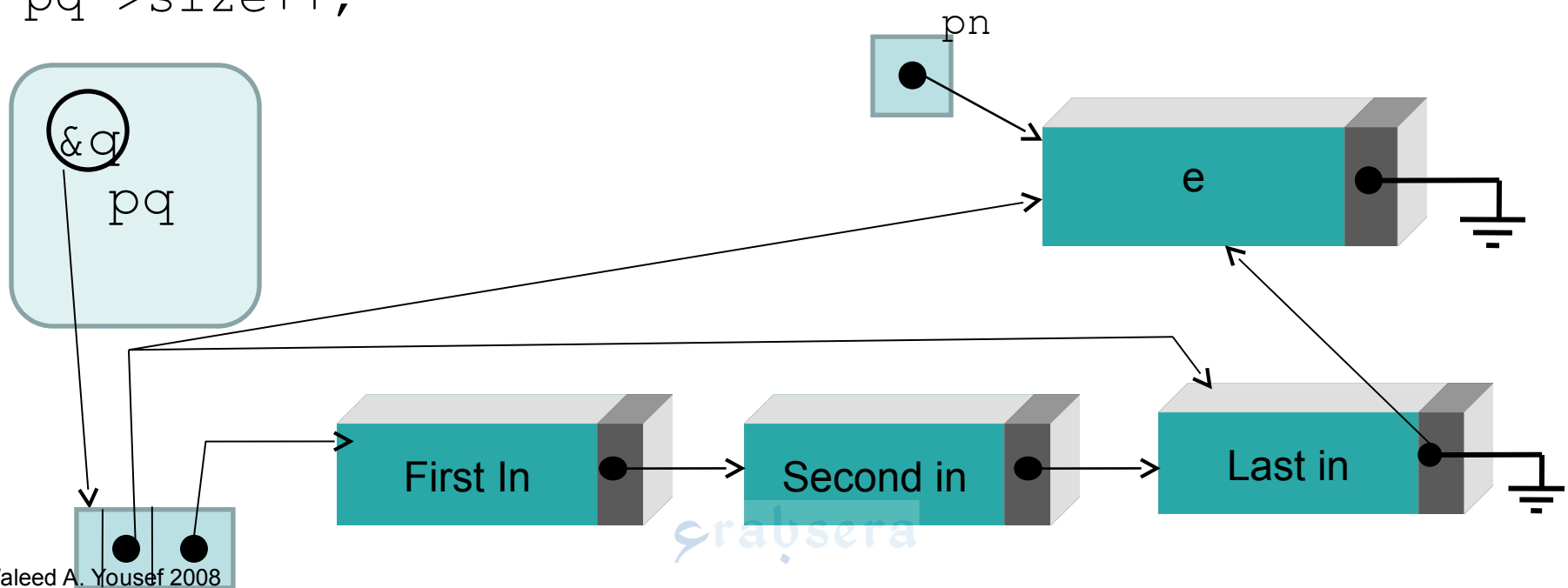

    pq->rear->next=pn;
  pq->rear=pn;
  pq->size++;
}
```

User Call:

```
Queue q;
QueueEntry e;

Append(e, &q);
```

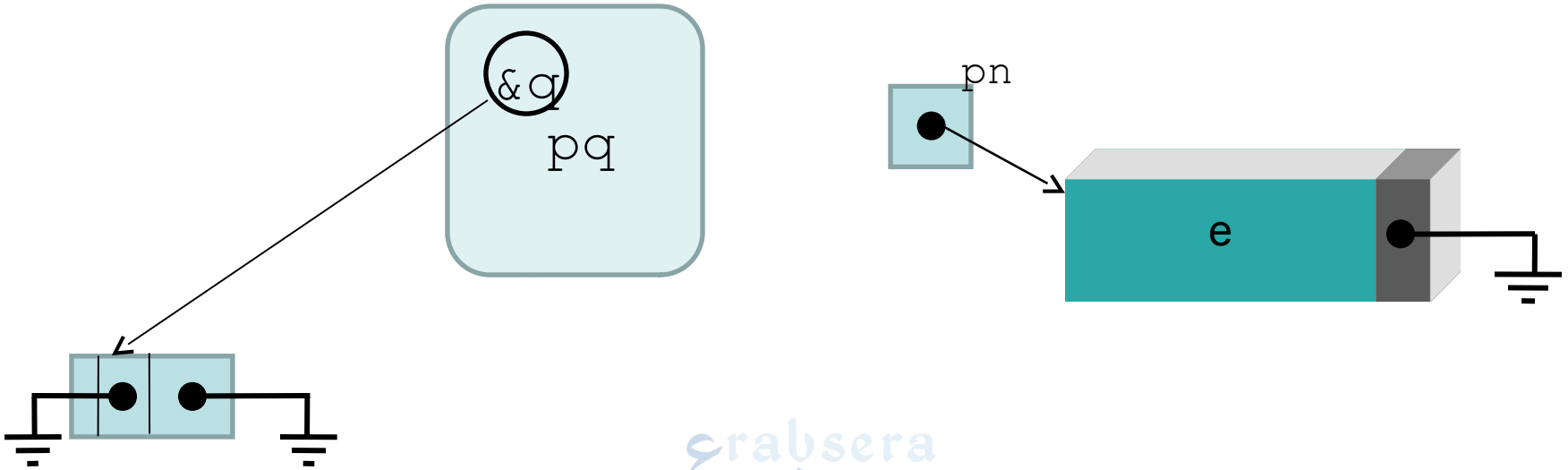




© Waleed A. Yousef 2008

Always take care of special cases (queue is empty)

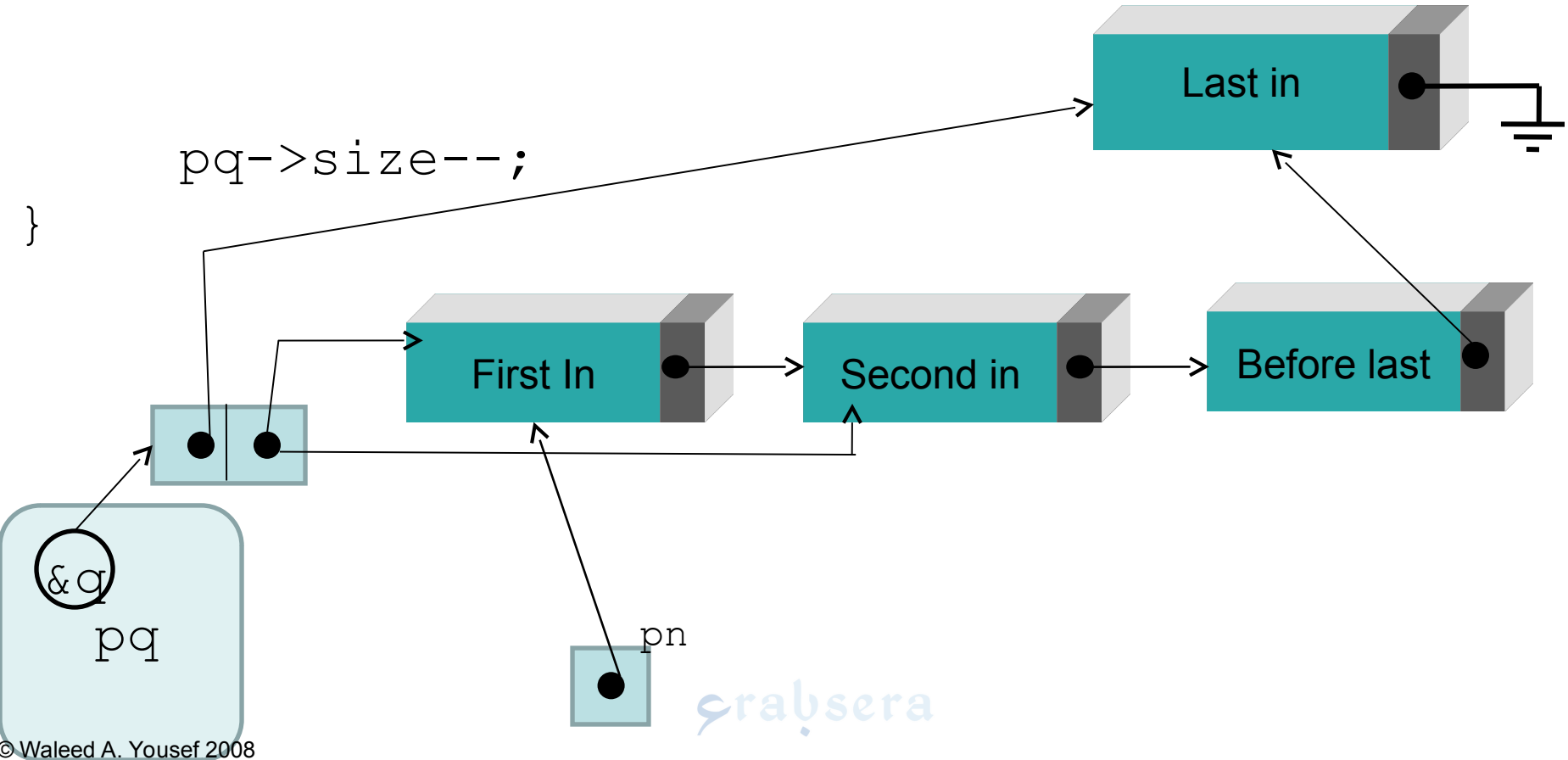
```
void Append(QueueEntry e, Queue* pq){
  QueueNode*pn=(QueueNode*)malloc(sizeof(QueueNode));
  pn->next=NULL;
  pn->entry=e;
  if (!pq->rear)
    pq->front=pn;
  else
    pq->rear->next=pn;//run time error for empty queue
  pq->rear=pn;
  pq->size++;
}
```



© Waleed A. Yousef 2008

```
void Serve(QueueEntry *pe, Queue* pq){
     QueueNode *pn=pq->front;
     *pe=pn->entry;
     pq->front=pn->next;
     free(pn);

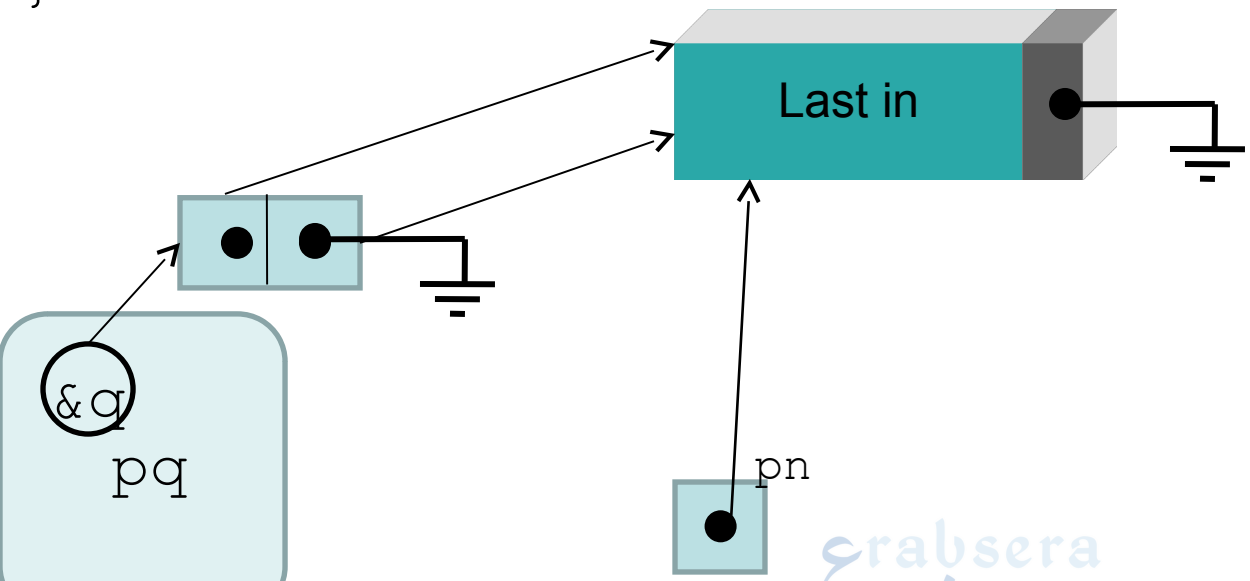

     pq->size--;
}
```



© Waleed A. Yousef 2008

Always take care of special cases: Only one element exists

```
void Serve(QueueEntry *pe, Queue* pq){
     QueueNode *pn=pq->front;
     *pe=pn->entry;
     pq->front=pn->next;
     free(pn);
     if (!pq->front)
          pq->rear=NULL;
     pq->size--;
}
```



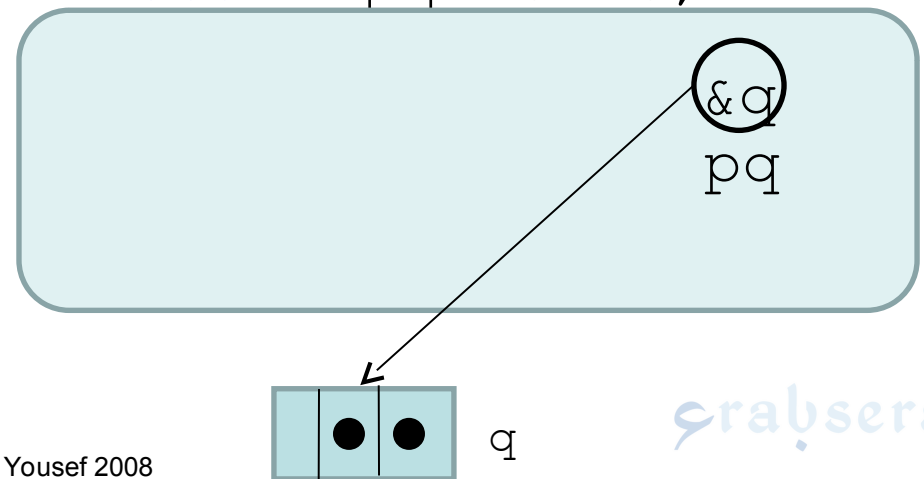
© Waleed A. Yousef 2008

Implementation level (what really happens)

```
int QueueEmpty(Queue* pq){
      return !pq->front;
}

int QueueFull(Queue* pq){
      return 0;
}

int QueueSize(Queue* pq){
      return pq->size;
}
```



User Level (interface)

```
void main(){

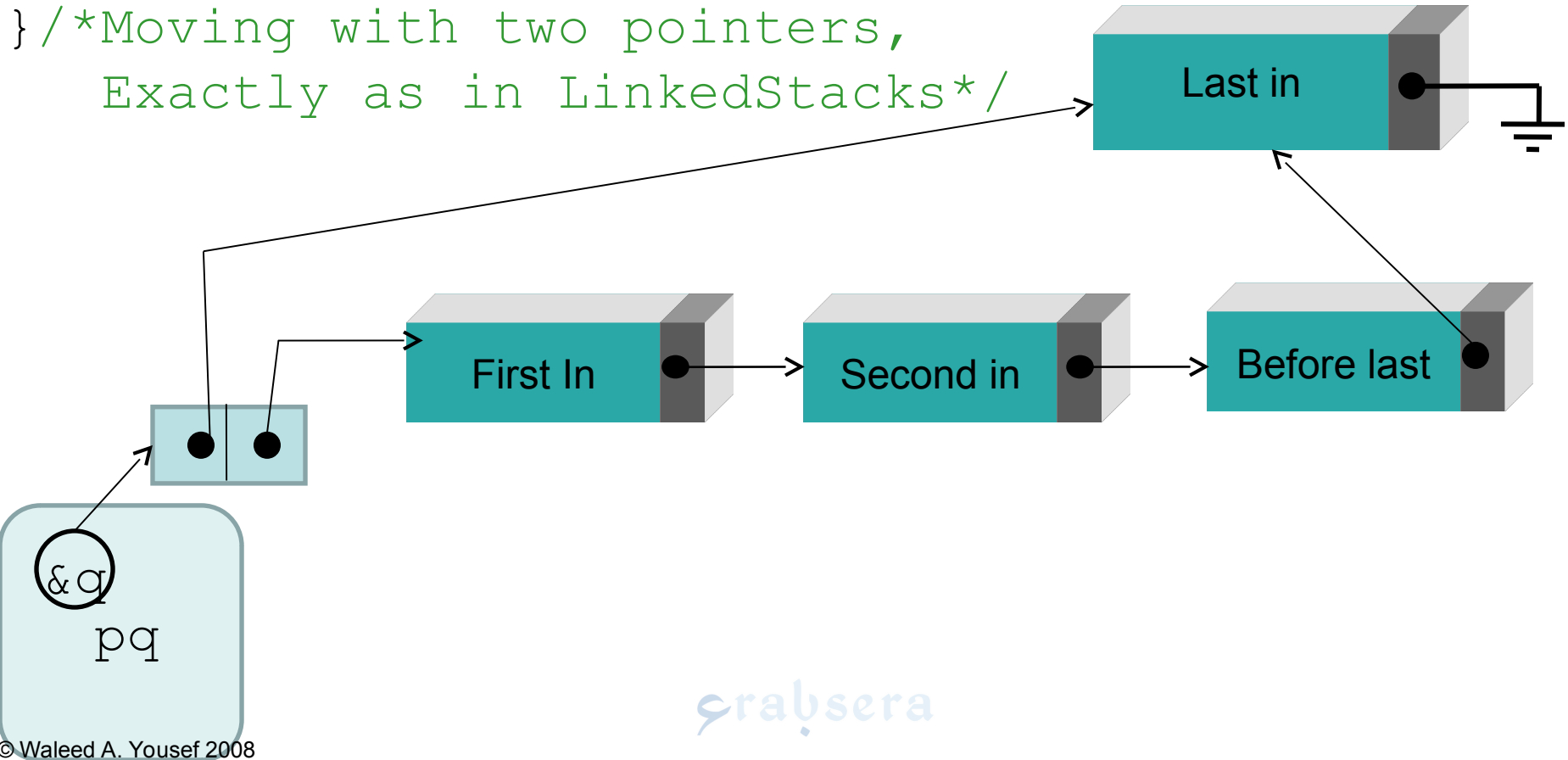
Queue q;

CreateQueue(&q);


}
```

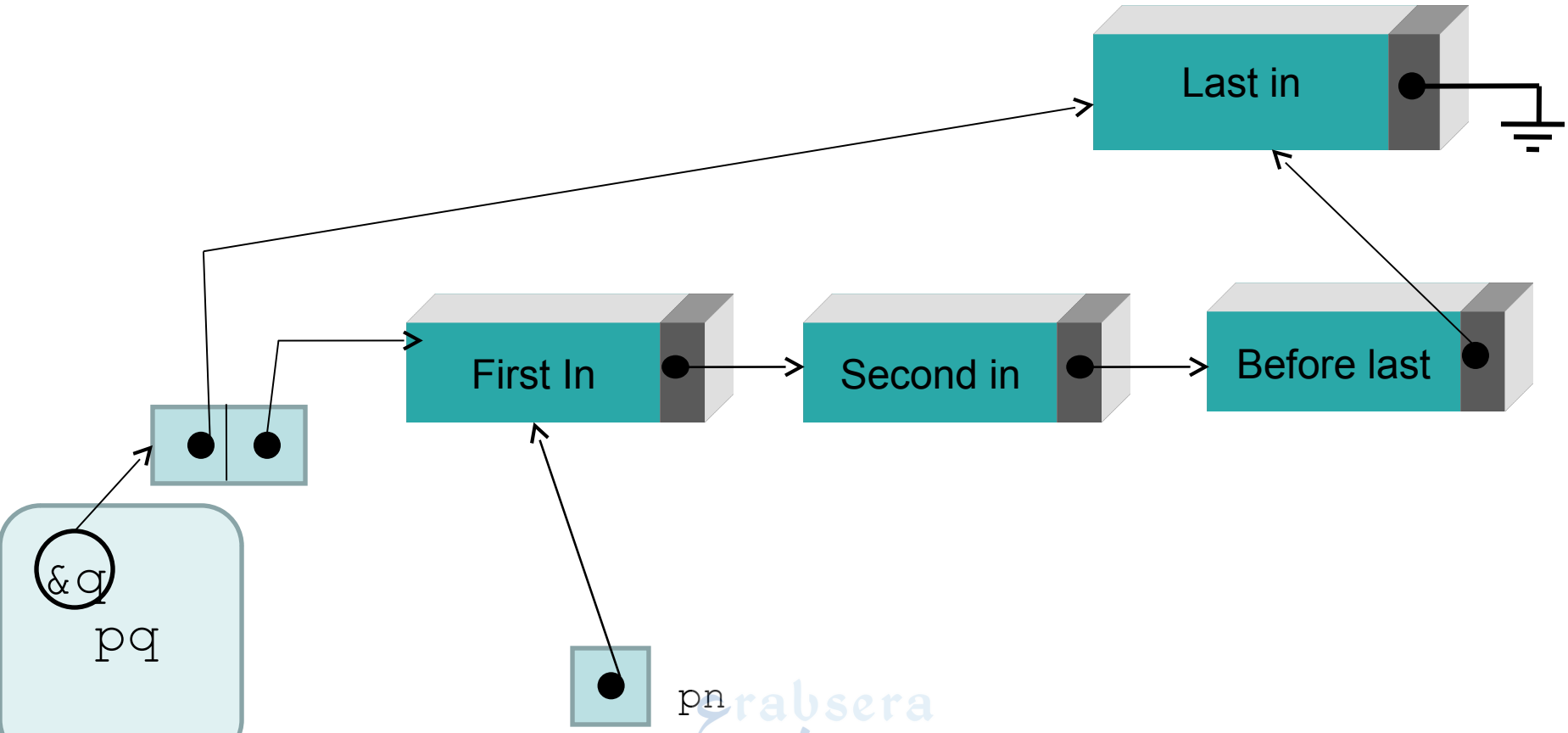
© Waleed A. Yousef 2008

```
void ClearQueue(Queue* pq){
     while(pq->front){
          pq->rear=pq->front->next;
          free(pq->front);
          pq->front=pq->rear;
     }
     pq->size  = 0;
}/*Moving with two pointers,
   Exactly as in LinkedStacks*/
```



© Waleed A. Yousef 2008

```
void TraverseQueue(Queue* pq, void(*pf)(QueueEntry)){
      for(QueueNode *pn=pq->front; pn; pn=pn->next)
            (*pf)(pn->entry);
}
```



© Waleed A. Yousef 2008

Very important note for all linked structures. E.g., in Queues:
In `Push` and `Append` we have to check for exhausted memory.
The code can be modified to:

```
int Append(QueueEntry e, Queue* pq){
  QueueNode*pn=(QueueNode*)malloc(sizeof(QueueNode));
  if (!pn)
    return 0;
    /*This is much better than the Error message of
      the book because this is more flexible. Also,
      the same function for contiguous implementation
      has to return 1 always to have consistent
interface*/
  else{
    //Put here exactly all of the remaining code
    return 1;
  }
```

```
...;
If (!Append(e, &q)){
      ...;
}
```

© Waleed A. Yousef 2008

Comparison between the array-based and the linked implementation: “Which is **always** better?” is a wrong question!

Do it yourself as a homework.

© Waleed A. Yousef 2008

Application of Queues: Simulation of an Airport Read it if you like

Simulation is the use of one system to imitate the behavior of another system. A ***computer simulation*** is a program to imitate the behavior of the system under study.

1. The same runway is used for both landings and takeoffs.
2. One plane can land or take off in a unit of time, but not both.
3. A random number of planes arrive in each time unit.
4. A plane waiting to land goes before one waiting to take off.
5. The planes that are waiting are kept in queues landing and takeoff, both of which have a strictly limited size.

© Waleed A. Yousef 2008